

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: Batch, Incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 7 - Processamento distribuído com Spark no Databricks

DataFrames, lazy evaluation, partições, joins, agregações e escrita Delta

I&D Capgemini Brasil, Setembro/2026

SPARK PARTITIONS & SHUFFLES

PARALELISMO, TROCA DE DADOS E DESEMPENHO



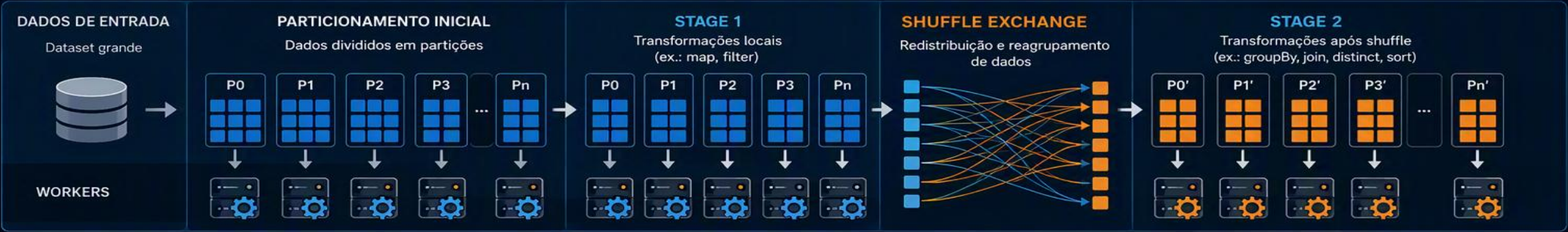
PARALELISMO
Dados divididos em partições executadas em paralelo nos workers.



SHUFFLE
Troca de dados entre stages para operações que exigem reagrupamento.



BALANCEAMENTO
Partições balanceadas garantem melhor uso de recursos e performance.



PARTIÇÕES BALANCEADAS (IDEAL)

- Dados distribuídos de forma uniforme.
- Todos os workers trabalham em ritmo semelhante.
- Menor tempo total de execução.

PARTIÇÕES DESBALANCEADAS (SKEW) ⚠️

- Dados concentrados em poucas partições.
- Alguns workers ficam sobrecarregados.
- Atraso na conclusão da job (straggler).

COMO OCORRE UM SHUFFLE

- Cada task grava os dados em arquivos de shuffle (partitioned by key).
- Os dados são transferidos pela rede para os reducers corretos.
- Reducer agrega e produz as novas partições para o próximo stage.

MAP TASKS (Stage 1) → **ARQUIVOS DE SHUFFLE (gravados localmente)** → **REDE (SHUFFLE)** → **REDUCER TASKS (Stage 2)**

1. WRITE → 2. TRANSFER → 3. READ & MERGE

OPERAÇÕES QUE CAUSAM SHUFFLE

- groupBy / aggregate
- join (exceto broadcast join)
- distinct
- sortBy / orderBy
- repartition / coalesce (com mudança de número de partições)

BOAS PRÁTICAS

- ✓ Escolha o número de partições adequado ao volume de dados e ao cluster.
- ✓ Evite shuffles desnecessários e colunas de alta cardinalidade como chave.
- ✓ Considere broadcast join para tabelas pequenas.
- ✓ Trate skew: salting, agrupar chaves quentes, ou AQE (skew join handling).
- ✓ Monitore no Spark UI: Stages, Tasks, Shuffle Read/Write e Data Skew.



Objetivos do Day 7

Sair do SQL/Data Mart e entrar no motor distribuído que executa em escala.

- Entender o papel do Apache Spark dentro do Databricks.
- Explicar DataFrames, transformações, ações e lazy evaluation.
- Relacionar partições, stages, tasks e shuffles ao desempenho.
- Implementar filtros, joins, aggregations e escrita Delta com PySpark.
- Reimplementar parte do pipeline anterior usando DataFrame API.

Mensagem central

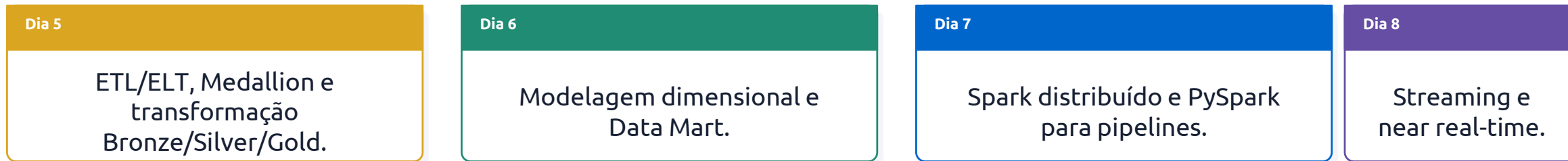
Spark não é “Python com dados grandes”. É execução distribuída orientada a plano.

Entregável

Notebook PySpark para transformação, comparando SQL e DataFrame.

Recap: onde o Day 7 entra na jornada

Depois de modelar Analytics, precisamos dominar o motor que transforma dados em escala.



- O Day 7 reforça a competência que sustenta transformações em grande escala.
- A mesma lógica de negócio pode ser escrita em SQL ou DataFrame API.
- O foco agora é entender como o processamento acontece no cluster.

Apache Spark no Databricks

Apache Spark no Databricks

Spark é a tecnologia que alimenta clusters de compute e SQL warehouses no Azure Databricks.

Compute

Clusters e SQL Warehouses executam workloads Spark.

APIs

SQL, DataFrames, PySpark, Scala e outras formas de programar.

Otimizações

Databricks Runtime adiciona otimizações sobre Apache Spark.

- Hoje, vamos usar PySpark DataFrames como linguagem principal.
- A plataforma abstrai parte da configuração do Spark, mas o engenheiro ainda precisa entender execução distribuída.
- Esse entendimento ajuda a depurar custo, performance e comportamento do pipeline.

DataFrame Transformations vs Actions in Spark

Transformations constroem um plano lógico (lazy). Ação dispara a execução do DAG no cluster.



TRANSFORMAÇÕES (Transformations)

Lazy Evaluation – constroem o plano lógico

Cadeia de transformações

```
df = spark.read.table("sales_raw")
    .filter("country = 'BR'")
    .select("order_id", "customer_id", "amount", "order_date")
    .withColumn("year", year("order_date"))
    .groupBy("customer_id", "year")
    .agg(sum("amount").alias("total_amount"))
    .orderBy(desc("total_amount"))
```



PLANO LÓGICO (Logical Plan)

Representação otimizada das transformações



Benefícios das Transformações

- ✓ Não executam imediatamente (lazy)
- ✓ Permitem otimizações do Catalyst Optimizer
- ✓ Podem ser combinadas e reordenadas
- ✓ Executam apenas quando uma ação é chamada



AÇÃO (Action)

Dispara a execução do plano e retorna um resultado

EXEMPLOS DE AÇÕES



show()



collect()



write()



count()



take()

...

outras



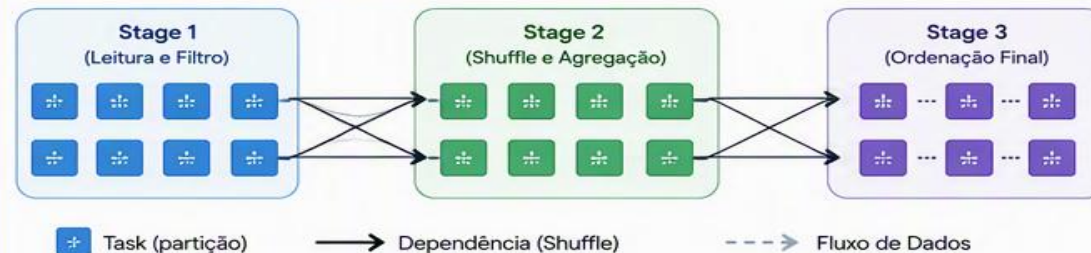
PLANO FÍSICO (Physical Plan)

Plano otimizado pronto para execução



DAG DE EXECUÇÃO (Execution DAG)

Tarefas executadas em paralelo no cluster



Resumo

Transformações constroem o plano lógico de forma preguiçosa. Uma ação materializa o resultado, executando o DAG no cluster.



Dica

Sempre encadeie transformações e execute ações apenas quando necessário para melhorar performance e reduzir custos.



Boas Práticas

- Filtre o quanto antes
- Selecione apenas as colunas necessárias
- Evite shuffles desnecessários
- Use partições adequadas ao volume de dados

Driver, executors, jobs, stages e tasks

Para entender Spark, pense em coordenação, paralelismo e fronteiras de shuffle.

Conceito	Função Didática
Driver	Coordena a aplicação e constrói o plano de execução
Executor	Executa tasks e processa partições de dados
Job	Trabalho disparado por uma action
Stage	Conjunto de tasks separado por fronteiras de shuffle
Task	Unidade de execução sobre uma partição

DataFrame API em PySpark

DataFrame é uma forma declarativa de transformar dados distribuídos.

DataFrame

Coleção distribuída organizada em colunas nomeadas.

Declarativo

Você descreve a transformação, Spark decide como executar.

Escalável

A mesma API opera sobre dados pequenos ou grandes.

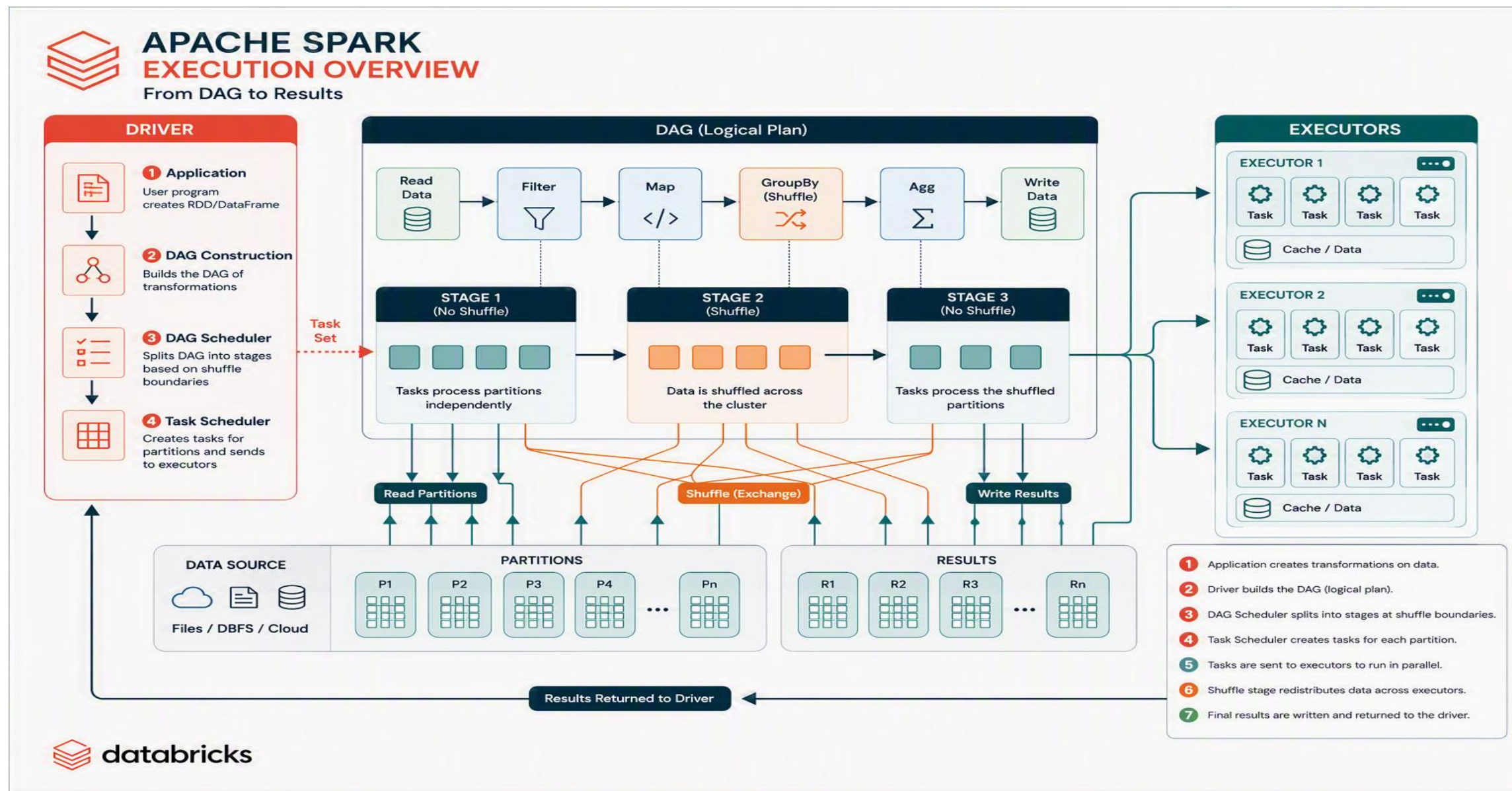
```
from pyspark.sql import functions as F

df1 = spark.table('capgemini_academy.gold.fat_vendas')

result = (df1
    .filter(F.col('ven_vl_total_pago') > 0)
    .groupBy('ven_sk_data_venda')
    .agg(F.sum('ven_vl_total_pago').alias('daily_revenue'))
)
```

Transformações vs Ações

Transformações constroem o plano; actions disparam a execução.



Lazy evaluation

Spark espera uma action para avaliar o plano completo e otimizar a execução.

Transformations

filter, select, join, groupBy e withColumn constroem o plano.

Actions

show, count, collect e write disparam execução.

Otimização

Spark enxerga o plano completo antes de executar.

```
df2 = (spark.table('capgemini_academy.bronze.sales')
      .filter(F.col('Order_Date') == '2026-06-07')
      .select('Order_ID', 'Customer_ID', 'Order_Value')
      .groupBy('Customer_ID')
      .agg(F.sum(F.col('Order_Value')).alias('total_revenue'))))

# Executa action
df2.write.mode('overwrite').saveAsTable('capgemini_academy.silver.customer_revenue')
```

Leitura de dados com PySpark

A leitura define a origem da transformação, mas ainda faz parte do plano lazy.

```
sales =  
spark.read.table('capgemini_academy.bronze.sales')  
  
customers =  
spark.read.table('capgemini_academy.bronze.customers')  
  
products =  
spark.read.table('capgemini_academy.bronze.products')  
  
sales.printSchema()  
sales.limit(5).display()
```

- Use nomes qualificados: catalog.schema.table.
- Confira schema e amostras antes de seguir.
- Evite display ou show em excesso em pipelines de produção.
- A leitura será combinada com filtros, joins e agregações no plano final.

Atenção

Em produção, ações de preview podem interromper o desenho otimizado do pipeline.

Filtros e seleção de colunas

Reduzir dados cedo melhora clareza e pode reduzir trabalho desnecessário.

```
sales_clean = (sales
    .filter(F.col('Shipping_Cost') > 0)
    .filter(F.col('Order_Date').isNotNull())

    .select('Order_ID', 'Customer_ID', 'Product_ID', 'Order_
Date', 'Quantity', 'Unit_Price')
    .withColumn('line_total', F.col('Quantity') *
F.col('Unit_Price'))
)
```

- Filtrar linhas inválidas antes de joins reduz volume.
- Selecionar apenas colunas necessárias simplifica o plano.
- withColumn cria colunas derivadas sem materializar imediatamente.
- O plano só será executado quando houver action.

Joins em Spark

Joins conectam datasets, mas podem gerar shuffle dependendo do tamanho e das chaves.

```
sales_enriched = (sales_clean
    .join(customers, on='Customer_ID', how='left')
    .join(products, on='Product_ID', how='left')
)
```

Regra prática

Antes do join, confira cardinalidade, duplicidade de chave e volume.

- Joins são essenciais para enriquecer fatos com dimensões.
- Chaves duplicadas podem multiplicar linhas sem erro explícito.
- Joins grandes geralmente são pontos de atenção para shuffle.
- Depois do join, valide contagem e amostra dos dados resultantes.

Aggregations

Agregações transformam linhas em métricas e normalmente exigem reorganização de dados.

```
daily_revenue = (sales_enriched
    .groupBy('order_date', 'city', 'category')
    .agg(
        F.sum('line_total').alias('revenue'),
        F.countDistinct('order_id').alias('orders'),
        F.sum('quantity').alias('units')
    )
)
```

- groupBy define a granularidade do resultado.
- Agregações são comuns no Silver -> Gold e em data marts.
- A mesma lógica pode ser expressa em SQL ou DataFrame API.
- Atenção a colunas de alta cardinalidade como chave de agrupamento.

Escrita em Delta

A action de escrita materializa o resultado no Lakehouse.

```
(daily_revenue.write
  .format('delta')
  .mode('overwrite')
  .option('overwriteSchema', 'true')
  .saveAsTable('capgemini_academy.gold.daily_revenue'
  ))
```

Ponto-chave

write é uma action: aqui o plano é executado de fato.

- Use Delta como formato padrão para tabelas governadas.
- Escolha modo de escrita conforme o cenário: overwrite, append ou merge.
- Após escrever, valide contagem, schema e amostra.
- A escrita fecha o ciclo read -> transform -> write.

Partições: paralelismo físico

Partição é a unidade de dados que permite execução paralela em tasks.

Mais partições

Mais paralelismo, mas também mais overhead de agendamento.

Menos partições

Menos overhead, mas maior risco de partições grandes.

Partições ruins

Podem gerar stragglers, spill e custo maior.

```
df.rdd.getNumPartitions()

# Ajuste físico, use com critério
df_repart = df.repartition(120, 'country')
df_small = df.coalesce(20)
```

Shuffle: quando Spark precisa mover dados

Shuffle é troca de dados entre executores/stages, comum em groupBy, join e distinct.

Operação	Tende a provocar shuffle?	Comentário
filter / select	Baixo	Geralmente narrow transformation
withColumn	Baixo	Depende da expressão
groupBy / agg	Alto	Reagrupa dados por chave
Join	Alto	Depende das tabelas e estratégia
Distinct	Alto	Precisa comparar chaves globalmente
orderBy	Alto	Ordenação global é cara

Transformações narrow vs wide

A diferença prática é se a transformação exige movimentação de dados entre partições.

Narrow transformation

Cada partição de saída depende de uma ou poucas partições de entrada.

Exemplos didáticos: filter, select, map simples.

Wide transformation

Partições de saída dependem de dados redistribuídos entre várias partições.

Exemplos didáticos: groupBy, join, distinct.

- Narrow transformations costumam escalar de forma mais direta.
- Wide transformations criam fronteiras de stage e podem gerar shuffle.
- Não evite wide transformations por regra; use quando a lógica de negócio exigir.
- O objetivo é reduzir shuffles desnecessários e validar chaves de alta cardinalidade.

Inspecionando o plano com explain()

Antes de otimizar, entenda o plano físico que Spark pretende executar.

```
daily_revenue.explain(mode='formatted')  
  
# Também útil durante investigação  
daily_revenue.explain(mode='extended')
```

- Use explain para identificar scans, filters, joins, aggregates e exchanges.
- Exchange é um sinal forte de shuffle no plano.
- Em investigação real, combine explain com Spark UI.
- A análise do plano ajuda a decidir se deve filtrar antes, ajustar join ou reduzir colunas.

Boa prática

Não optimize no escuro. Primeiro observe o plano e as métricas de execução.

Spark UI: o painel para entender execução

A UI ajuda a olhar jobs, stages, tasks, duração e sinais de shuffle.

Jobs	Stages	Tasks
O que foi disparado por actions.	Fronteiras de execução, geralmente influenciadas por shuffle.	Execuções por partição, úteis para identificar skew.

- Procure stages muito mais lentos que os demais.
- Compare duração das tasks dentro do mesmo stage.
- Observe shuffle read/write quando houver joins e aggregations.
- Use a UI como evidência antes de alterar código ou cluster.

Noções de performance para Engenheiros de Dados

Performance começa com desenho de pipeline, não só com cluster maior.

Prática	Por que importa
Filtrar cedo	Reduz volume processado nas próximas etapas
Selecionar colunas	Evita carregar dados desnecessários
Validar chaves de join	Evita explosão de cardinalidade
Cuidado com groupBy	Pode gerar shuffle pesado
Evitar actions intermediárias	Preserva otimização do plano
Observar Spark UI	Evita otimização baseada em chute

SQL vs DataFrame API

O objetivo do laboratório é comparar duas formas de expressar a mesma transformação.

SQL

Ótimo para lógica declarativa, agregações, joins e leitura por analistas.

DataFrame API

Boa para composição programática, funções reutilizáveis e pipelines em Python.

```
# SQL
```

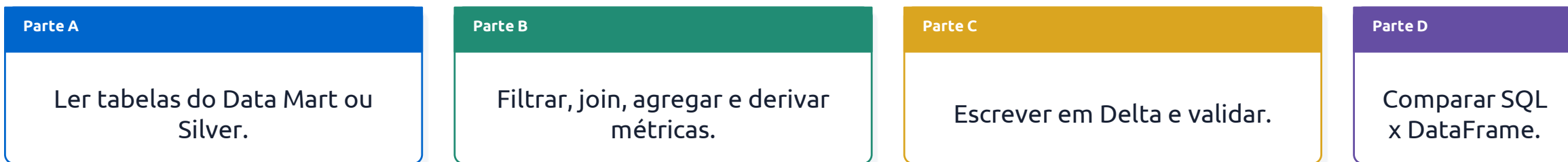
```
spark.sql('SELECT product_id, SUM(order_value) AS revenue FROM sales GROUP BY product_id')
```

```
# DataFrame
```

```
sales.groupBy('product_id').agg(F.sum('order_value').alias('revenue'))
```

Laboratório: visão geral

Reimplementar parte do pipeline anterior em PySpark e comparar com SQL.



- Escolha uma transformação do Dia 6, como receita por mês/categoria ou segment de produto.
- Implemente primeiro em SQL, depois em PySpark DataFrame.
- Compare legibilidade, reuso, plano de execução e validações.

Setup do notebook PySpark

Comece com imports, parâmetros e nomes qualificados.

```
from pyspark.sql import functions as F

catalog = 'capgemini_academy'
schema = 'gold'
source_table = f'{catalog}.{schema}.fat_vendas'
product_table = f'{catalog}.{schema}.dim_produtos'
date_table = f'{catalog}.{schema}.dim_datas'
target_table = f'{catalog}.{schema}.revenue_by_month_category_pyspark'

fato = spark.table(source_table)
produtos = spark.table(product_table)
datas = spark.table(date_table)
```

- Use parâmetros para facilitar promoção entre ambientes.
- Evite caminhos hardcoded quando a tabela já está governada em catálogo/schema.
- Mantenha o notebook legível: setup, transformação, validação e escrita.

Atividades práticas

Bootcamp Databricks

I&D Capgemini Brasil

Bootcamp Databricks

Laboratório Dia 7

Processamento distribuído com Spark e DataFrame API

Objetivo: Reimplementar uma transformação do Data Mart do Dia 6 usando PySpark DataFrame API, comparando com SQL e observando lazy evaluation, joins, agregações, shuffle, explain e escrita Delta.

Narrativa: até o Dia 6 criamos o modelo analítico. No Dia 7 aprendemos como o Spark executa essa lógica em escala.

Jornada prática do laboratório

1

GOLD

Reusar Gold

Carregar fato e dimensões do catálogo capgemini_academy.

2

SQL

SQL referência

Executar receita por período/produto em SQL para ter resultado esperado.

3

DF

DataFrame API

Recriar a mesma lógica com joins, groupBy e agg em PySpark.

4

LAZY

Lazy evaluation

Mostrar que transformações montam plano; actions executam.

5

PLAN

explain()

Inspecionar o plano e localizar scans, joins, aggregates e Exchange.

6

SHUF

Shuffle

Relacionar groupBy e join com wide transformations e movimentação de dados.

7

DELTA

Escrita Delta

Materializar resultado em gold.revenue_by_month_pyspark.

8

QA

Validação

Comparar SQL x PySpark: contagem, totais e nulos críticos.

Base reaproveitada

capgemini_academy.gold

FACT

fact_vendas
medidas

DIMENSÕES

dim_datas
dim_produtos

RESULTADO

revenue_by
_month_pyspark

SQL + PySpark

Checklist de validação

✓

Tabelas do Dia 6 reutilizadas

✓

Resultado SQL gerado como baseline

✓

DataFrame API implementada

✓

explain() executado e comentado

✓

Exchange/shuffle identificado

✓

Resultado final escrito em Delta

Comandos centrais

```
fact = spark.table("capgemini_academy.gold.fact_sales")
date = spark.table("capgemini_academy.gold.dim_date")
result = fact.join(date, "date_key")
            .groupBy("ano", "mes").agg(F.sum("valor_total"))
result.explain(mode="formatted")
result.write.mode("overwrite").saveAsTable(target)
```

Critério: o resultado PySpark deve bater com o SQL baseline.

Spark não executa instruções. Spark executa planos.

Lab: transformação com DataFrame API

Join + agregação + ordenação em PySpark.

```
result = (fato
    .join(produtos, on=[fato['ven_sk_produto'] ==
produtos['pro_sk_produto']], how='left')
    .join(datas, on=[fato['ven_sk_data_venda'] ==
produtos['dat_sk_data']], how='left')

    .groupBy('dat_nu_ano', 'dat_nu_mes', 'pro_ds_categoria')
    .agg(
        F.sum('ven_vl_total_pago').alias('revenue'),
        F.countDistinct('ven_id_venda').alias('orders'),
        F.sum('ven_qt_itens').alias('units')
    )
    .withColumnRenamed('dat_nu_ano', 'ano')
    .withColumnRenamed('dat_nu_mes', 'mes')
    .withColumnRenamed('pro_ds_categoria',
'categoria')
    .orderBy('ano', 'mes', F.desc('revenue'))
)
```

- A transformação é legível como pipeline encadeado.
- Cada etapa retorna um novo DataFrame.
- A execução real ocorre na escrita ou em outra action.
- Use explain para observar o plano antes de materializar se necessário.

Lab: escrita e validação

Materialize o resultado em Delta e valide antes de encerrar.

```
(result.write
  .format('delta')
  .mode('overwrite')
  .option('overwriteSchema','true')
  .saveAsTable(target_table))

# Validações básicas
spark.table(target_table).count()
spark.table(target_table).filter(F.col('revenue').isNull()).count()
```

- Valide contagem, nulos e totais principais.
- Compare resultado com a versão SQL quando possível.
- Documente a diferença entre o resultado esperado e o realizado.
- O entregável é o notebook PySpark de transformação.

Comparação: SQL e DataFrame

A comparação deve ser técnica, não preferência pessoal.

Critério	SQL	DataFrame API
Leitura por analistas	Alta	Média
Composição programática	Média	Alta
Reuso em funções	Médio	Alto
Expressividade para joins/aggs	Alta	Alta
Debug em Python	Médio	Alto
Integração com libs Python	Baixa	Alta

Mini desafio em aula

Reimplemente uma transformação do Dia 6 e responda tecnicamente.

- Qual é a granularidade do resultado gerado?
- Quais operações provavelmente geraram shuffle?
- O resultado bate com a versão SQL?
- O plano contém Exchange? Onde?
- Qual validação impediria um erro silencioso de cardinalidade?

Objetivo do desafio

Transformar o aluno de usuário de Spark em engenheiro que raciocina sobre execução.

Entrega

Notebook com código, explain, validações e comentários.

Erros comuns no primeiro PySpark em produção

A maioria dos problemas está no modelo mental, não na sintaxe.

Erro 1

Usar show/count demais e materializar no meio do pipeline.

Erro 2

Fazer join sem validar cardinalidade da chave.

Erro 3

Ignorar partições, shuffle e Spark UI.

- Ajuste o código com base em evidência: plano, métricas e validações.
- Não troque SQL por PySpark apenas por estética. Use PySpark quando houver ganho de composição, reuso ou integração.
- Não escale cluster para compensar desenho ruim do pipeline.

Ponte para o Day 8: streaming

Agora que entendemos batch distribuído, o próximo passo é processamento contínuo.

Day 7

Batch processing com PySpark e Spark distribuído.

Day 8

Streaming vs batch, eventos, janelas e late events.

Continuidade

Mesma base Spark, outro modelo de execução.

- O batch processa conjuntos delimitados de dados.
- O streaming processa dados continuamente ou em micro-batches.
- O raciocínio sobre plano, estado e checkpoints será essencial no próximo dia.

Fim do Day 7



Desafio do Dia 7 – Assistir e resumir

- O que é Spark? (Explicação visual)

<https://www.youtube.com/watch?v=nKRwjXRs4mY&t=26s>

Thank you.

Make it real.

Capgemini 